

Programming Fundamentals (COS2531)

Assignment 1

Assessment Type	Individual assignment (no group work). Submit online via Canvas/Assignments/Assignment 1. Marks are awarded per rubric (please see the rubric on Canvas). Clarifications/updates may be made via announcements. Questions can be raised via the lectorial, practical sessions or Canvas discussion forum. Note the Canvas discussion forum is preferable.
Due Date	End of Week 6 (exact time is shown in Canvas/Assignments/Assignment 1) Deadlines will not be advanced, but they may be extended. Please check Canvas/Assignments/Assignment 1 for the most up-to-date information regarding the assignment. As this is a major assignment, a university standard late penalty of 10% per day (e.g., 2 marks/day) applies, unless extension request or special consideration has been granted.
Weighting	20 marks out of 100

1. Overview

The objective of this assignment is to develop your programming and problem-solving skills in a step-by-step manner. The different stages of this assignment are designed to gradually introduce different basic programming concepts.

You should develop this assignment in an iterative fashion (as opposed to completing it in one sitting). You can and should get started now (when this assignment specification is posted on Canvas) as there are concepts from previous lessons that you can employ to do this assignment. If there are questions, you can ask via the lectorial, practical sessions or the Canvas discussion forum (Canvas/Discussions/Discussion on Assignment 1). Note that the **Canvas discussion forum is preferable** as it allows other students to see your questions as well. Also, you should ask questions in a general manner, for example, you should replicate your problem in a different context in isolation before posting, and **you must not post your code on the Canvas discussion forum**.

2. Assessment Criteria

This assignment will determine your ability to:

- i. Follow coding, convention and behavioural requirements provided in this document and in the course lessons;
- ii. Independently solve a problem by using programming concepts taught over the first several weeks of the course;
- iii. Write and debug Python code independently;
- iv. Document code;
- v. Provide references where due;
- vi. Meet deadlines;
- vii. Seek clarification from your "supervisor" (instructor) when needed via the Canvas discussion forums; and
- viii. Create a program by recalling concepts taught in class, understand, and apply concepts relevant to solution, analyse components of the problem, and evaluate different approaches.

This assignment is relevant to the following Learning Outcomes:

1. Analyse simple computing problems.
2. Devise suitable algorithmic solutions and code these algorithmic solutions in a computer programming language.

Specifically, upon the completion of this assignment, you will be able to:

- Demonstrate knowledge of basic concepts, syntax, and control structures in programming
- Devise solutions for simple computing problems under specific requirements
- Encode the devised solutions into computer programs and test the programs on a computer
- Demonstrate understanding of standard coding conventions and ethical considerations in programming

4. Assessment Details

Please ensure that you have read Sections 1-3 of this document before going further.

Problem Overview: You are to design and implement a simple console-based application called “Football Stats Manager”. The program helps a competition organiser to:

- Create group tournaments for football (soccer) teams.
- Register teams into groups.
- Create and view group matches.
- Record match results and automatically update group tables (wins, draws, losses, goals for/against, points).
- Determine the top 2 teams from each group to go to next round

Requirements: Your code must meet the following **functionalities**, **code**, and **documentation** requirements. Your submission will be graded based on the **rubric** published on Canvas. Please ensure you read all the requirements and the rubric carefully before working on your assignment.

A - Functionalities Requirements:

There are 2 parts; please ensure you only attempt one part after completing the previous part.

----- PART 1 (6 marks) -----

In the first part, you will cover the minimum viable features of the program, which is to create group stats table based on a match outcome entered by user. You will extend this core feature further to improve its interactivity and provide more beneficial features in Part 2.

The first step is to focus on collecting inputs about a particular match. The following information will be recorded: group name, home team name, away team name, home team goal and away team goal with the following details:

- Group name can be any string, a typical group name in a match is a capitalized letter of the alphabet.
- Match number which should be a whole number.
- Home team name is the name of the first team in the match, and it can contain few words.
- Away team name is the name of the second team in the match, and it can contain few words.

After the input, the match summary will be printed, and the winner will be determined based on the number of goals or inform of a draw if home and away team goals were equal.

Group name: A

Match number: 1

Home team name: Brazil

Away team name: Argentina

Home team goal: 3

Away team goal: 0

Winner of match 1 is Brazil.

Based on the given information, a team’s record will be collected on a table for each group.

To understand how football statistics work here is list of fields used to record each team’s performance accumulated after a typical match.

Term	Abbreviation	Description
Matches played	MP	Number of matches this team has had.
Wins	W	The number of matches this team has won.
Lose	L	The number of matches this team has lost.

Draw	D	The number of matches resulted in a draw.
Goals for	GF	Total goals this team has made.
Goals against	GA	Total goals this team's opponent has against this team.
Goals difference	GD	Total goal difference against all opponents, calculated using the following formula: $GD = GF - GA$
Points	Pts	Points collected by a team and calculated using the following formula: $Pts = (3 \times W) + D$

Output example based on the given input would be as follows:

Group A

Brazil vs Argentina

Brazil's stats

MP = 1 *it's their first match, so 1 each side*

W = 1 *to be determined by comparing the GF vs GA*

D = 0 *to be determined by comparing the GF vs GA*

L = 0 *to be determined by comparing the GF vs GA*

GF = 3 *as per input*

GA = 0 *as per input*

GD = 3 *apply the formula: $GD = GF - GA = 3 - 0 = 3$*

Pts = 3 *apply the formula: $Pts = (3 \times W) + D = (3 \times 1) + 0 = 3$*

Argentina's stats

MP = 1

W = 0

D = 0

L = 1

GF = 0

GA = 3

GD = -3 *apply the same formula: $0 - 3 = -3$*

Pts = 0 *apply the same formula: $(3 \times 0) + 0 = 0$*

After recording the match, each team's stats should be treated like a row in the table. The program needs to collect this information in a list of rows (effectively forming a table) that can be recalled later.

The next part is to be able to recall the table for a particular group by prompting a group name.

The following information will be displayed in a tabular format as such:

Team	MP	W	D	L	GF	GA	GD	Pts
Brazil	1	1	0	0	3	0	3	3
Argentina	1	0	0	1	0	3	-3	0

----- PART 2 (9 marks, please do not attempt this part before completing PART 1) -----

In this part, your program can: (a) perform some additional requirements compared to PART 1, and (b) operate using a menu.

For marking and backup purpose, before starting Part 2 please bundle Part 1 code segment into a function called `part_1()` that can be run from the main segment at the bottom the file.

Your program will now operate using a menu. A menu-driven program is a computer program in which options are offered to the user via a menu. Your program will have the following options:

1. Create group tournaments. This is to register all participants and place them in a group of four teams. The number of groups will depend on the number of participants and each group will be given a unique name.
 - a. Prompt the user for the total number of participating teams.
 - b. Validate that the number of teams is divisible by 4.
 - c. Determine the number of groups (each group contains 4 teams).
 - d. For each group:
 - i. Ask for a **group name** (e.g., “Group A”).
 - ii. Ask for and store **exactly 4 team names** in that group.
 - e. Maintain:
 - i. A list of group records (or dictionaries).
 - ii. An initial “table” (statistics) for each team (e.g., matches played, wins, draws, losses, goals for, goals against, points).
2. View the tournament. This is to display all groups and their comprising teams.
 - a. Display all groups and the teams registered in each group.
 - b. Display the total number of groups and teams.
3. Create group matches. After teams listed and allocated in groups, matches need to be scheduled in round-robin format, which means each team plays against the others once. Each team will only play with other teams within the same group and not those that belong to another group.
 - a. Allow the user to select a group by name.
 - b. Generate or input the matches between the 4 teams in that group.
 - c. Store each match with:
 - i. Group name
 - ii. Home team (Team 1)
 - iii. Away team (Team 2)
 - iv. (Automatically) a match ID

Prompt an option to create a group match for another group and repeat the creation process if agreed or go back to the main menu if not.
4. View group matches. This is to display team match schedules within a group, including the two competing team names in each match and a match ID.
 - a. Allow the user to enter a group name.
 - b. Display all matches scheduled for that group.
5. Record match scores. Allow the user to select a match (e.g., by group name and teams, or by match ID).
 - a. Prompt for goals scored by each team.
 - b. Print the match summary in the same format as Part 1.
 - c. Update team statistics in that group’s table:
 - i. Matches played (MP)
 - ii. Wins (W), Draws (D), Losses (L)
 - iii. Goals For (GF), Goals Against (GA), Goals Difference (GD)
 - iv. Points (e.g., Win = 3 pts, Draw = 1 pt, Loss = 0 pts)
 - d. Use a function called `calculate_points_and_stats` to encapsulate the logic for updating a team’s record.
6. View group tables (**Special challenge, you may use Val**)

This part is worth a higher mark and requires additional research beyond the course’s

materials. You will need to investigate how to sort data. In your implementation, you may use a list of lists or list of dictionaries or any combination of basic data structures without using any additional module. Please investigate the various sorting methods available for the data structures of your choice and justify why you had chosen that method in your code comments.

- a. For any prompted and given group, display the group table showing all team records including Team name, MP, W, D, L, GF, GA, GD, and Pts.
 - b. Sort the table by points in descending order.
 - c. Further sort the table by goal difference, and goals for; this additional sort will determine who is ahead in case of a draw where 2 or more teams got the same points.
7. Determine winners (**Special challenge, you may use Val**)
This is to conclude group stage outcomes and determine top 2 teams from each group that got through to the next stage in the tournament known as the knockout stage. This can only work every team in each group already played against every other team, i.e. played 3 matches. If group matches are ongoing, display appropriate messages for it. Once group stage is concluded, display a list of winners from each group. Only the top two will go to the next tournament stage. Please collect and display them all on one list. Please note that the knockout stage is out of scope for this assignment.
8. Exit. Terminate the application gracefully with an appropriate message.

Your program will be able to handle **invalid user inputs**. The following sensible inputs are expected, and prompt is to be repeated until the valid input is given:

- a. Number of participants must be divisible by 4. This input must be validated correctly, and you cannot have any group with less than or more than 4 participants.
- b. Group names can comprise many words, including a combination of letters and numbers.
- c. Team names cannot contain numbers.
- d. Goals must be a whole number and cannot be negative.
- e. A match ID must be numerical and need to be unique per group, e.g. it is possible to have group A match #1, group B match #1, and group B match #1, but it is incorrect to have identical match ID in the same group twice.

Note that in your program, when a task (option) is accomplished, the menu will appear again for the next task.

You may only use Val assistance for help solving special challenges specified in menu items 6 and 7. You may not use Val or any other Gen AI for assistance in working on other parts of the assignment. When in doubt, your assessor may organise an additional Q&A session to determine your understanding of your own work.

B - Code and Technical Requirements:

The program **must be entirely in one Python file named ProgFunA1_Your Student ID.py**. For example, if your student ID is s1234567, then the Python file must be named ProgFunA1_s1234567.py. Other names will not be accepted.

Your code needs to be formatted consistently. You must not include any unused/irrelevant code (even inside the comments). Use meaningful variables and function names. Comment on your code to explain non-obvious logic. Maintain consistent indentation and formatting. What you submitted must be considered as the final product.

You should use appropriate data types and handle user inputs properly. You must not have any redundant parts in your code. For user interaction, please use clear console prompts and messages to ensure the menu repeats until the user chooses to exit.

You must demonstrate your ability to program in Python by yourself, i.e., you should not attempt to use external special Python packages/libraries/classes that can do most of the coding for you. **The only Python library allowed in this assignment is the sys module.**

Note that in places where this specification may not tell you how exactly you should implement a certain feature, you need to use your judgment to choose and apply the most appropriate concepts from our course materials. You should follow answers given by your "client" (or "supervisor" or the teaching team) under Canvas/Discussions/Discussion on Assignment 1.

C - Documentation Requirements:

You are required to write comments as a part of your code (i.e. in-code comments). Writing comments is a good habit in professional programming. It is particularly useful if the comments are next to the code segment that it refers to. Note that you don't need to write an essay, i.e., you should keep the comments succinctly.

At the beginning of your Python file, your code must contain the following information:

1. **Your name and student ID.**
2. **The highest part you have attempted.**

Besides, you are required to provide **design justifications** for your program. The purpose of this section is to explain the reasoning behind your design decisions, rather than restate the Python code line by line. Your design justifications should address the following points:

- **Design decisions and rationales:** Explain the key design decisions you made and why you made them, as well as lessons learned during development. For example, you may discuss why you introduce a new function or why you choose certain control structures, or why you select specific data types or data structures to store information, and what you have learned through the development.
- **Special challenges:** If you attempted the special challenges specified in menu 7 and 8, you should provide explanation and justification of your algorithm. If you decide to use GenAI like RMIT Val as a reference, you must clearly declare it in your source code. You must also provide a critique of what Val has suggested and how its suggestions were suitable or not suitable for your purpose.
- **Challenges and limitations:** Describe any challenges you encountered while developing the program and explain how you addressed them. You should also clearly describe any known limitations (e.g. scenarios that might cause the program to crash or behave abnormally), or requirements that your program does not fully satisfy.
- **References:** You must acknowledge any external sources of information used (outside of the course materials provided under Canvas/Modules, and Python official documentation/tutorials). For each source, you need to briefly explain how it was used in your assignment. More detailed information regarding the references can be found in Section 7.

Note design justifications should be included in the same Python file (your Python submission), preferably placed at the end of the file. Please DO NOT submit a separate file for this.

D - Rubric:

Overall:

Part	Points
Part 1	6
Part 2	9
Design Justification	4
Others (code quality, modularity, comments)	1

More details of the rubric of this assignment can be found on Canvas A1 instruction page. Students are required to look at the rubric to understand how the assignment will be graded.

5. Example Program

The following is a **sample program** that satisfies the requirements specified in Section 4. Note that this is just an example, so it is okay if your program looks slightly different, but you need to make sure that **your program satisfies the requirements listed in Section 4**.

5.1. PART 1

As examples, these are how the output screens of our sample program look like for PART 1. In below example, a match happened between Australia and Spain. Note that in this test, we use the values sample data only. You should test your program with different test cases, to make sure your program satisfies the requirements in Section 4.

```

Enter a group name: A
Enter a match number: 2
Enter a home team: Australia
Enter a away team: Spain
Enter a home team goals: 1
Enter a away team goals: 0

Group A Match 2 summary
-----
Australia 1 vs Spain 0
Winner of match 2 is Australia
  
```

In the example below, group table got calculated and displayed. Note that in this test, we use the values Australia (1) and Spain (0) sample data only as per above. You should test your program with different test cases, to make sure your program satisfies the requirements in Section 4.

Team	MP	W	D	L	GF	GA	GD	Pts

Australia	1	1	0	0	1	0	1	3
Spain	1	0	0	1	0	1	-1	0

5.2. PART 2

As an example, this is how the output screen of our sample program looks like for a menu with all the options described in PART 2.

```
Football Stats Manager
Menu:
1. Create group tournaments
2. View a group tournament
3. Create group matches
4. View group matches
5. Record match scores
6. View group tables
7. Determine winners
8. Exit
Enter your choice: 1
Create group tournaments
Enter number of teams participating: 8
Number of groups based on teams 2
Group 1 of 2
Enter group name: A|
```

When the user enters an option, the corresponding functionality will appear. For example, if the user chooses option 1, which is to perform a service, then the output screen of our sample program is as follows.

```
Enter your choice: 1
Create group tournaments
Enter number of teams participating: 8
Number of groups based on teams 2
Group 1 of 2
Enter group name: A
You will register 4 teams for Group A
Enter team name: Mexico
Enter team name: South Africa
Enter team name: South Korea
Enter team name: Czechia
Group 2 of 2
Enter group name: B
You will register 4 teams for Group B
Enter team name: Switzerland
Enter team name: Canada
Enter team name: Bosnia
Enter team name: Qatar
```

Other requirements in PART 2 can also be displayed in a similar manner.

6. Submission

As mentioned in the Code Requirements, **you must submit two files named ProgFunA1_YourStudentID.py and Condition3_Bound_Process_AI_SubmissionDoc_YourStudentID.pdf** via Canvas/Assignments/Assignment 1. **It is your responsibility to correctly submit your file.** Please verify that your submission is correctly submitted by downloading what you have submitted to see if the file includes the correct content. The final .py file submitted is the one that will be marked.

Late Submission

All assignments will be marked as if submitted on time. Late submissions of assignments without special consideration or extension will be automatically penalised at a rate of 10% of the total marks available per day (or part of a day) late. This includes cases where the assessment work was uploaded earlier but the required declaration was not submitted on time.

Late penalties will apply from the original due date, unless you have an approved extension or special consideration.

For example, if an assignment is worth 20 marks and it is submitted 1 day late, a penalty of 10% or 2 marks will apply. This will be deducted from the assessed mark.

Special Consideration

If you are applying for extensions for your assessment within five working days after the original assessment date or due date has passed, or if you are seeking extension for more than seven days, you will have to apply for Special Consideration, unless there are special instructions on your Equitable Learning Plan.

In most cases you can apply for special consideration online [here](#). For more information on special consideration, visit the university website on special consideration [here](#).

7. Referencing Guidelines

What: This is an individual assignment, and all submitted contents must be your own. If you have used any sources of information (e.g., websites, tools) other than the course contents directly under Canvas/Modules, **you must give acknowledgement of the sources, explaining in detail how you use the sources in this assignment, and give references using the [IEEE referencing format](#).**

Where: You can add a code comment near the work (e.g., code block) to be referenced and include the detailed reference in the IEEE style.

How: To generate a valid IEEE style reference, please use the [citethisforme](#) tool if you're unfamiliar with this style, alternatively you can also go to RMIT Library EasyCite.

This assignment is intended to be assessed according to Condition 3 of the Gen AI Assessment framework, which is bounded process with AI.

You must **submit the Condition 3 Bounded Process AI Declaration** with your assessment, **even if you did not use AI. If you did not use AI, complete the section confirming that no AI was used.**

If you decide to use RMIT Val which is the only endorsed Gen AI tool in this course as a reference, you must clearly declare it in your source code. You must also provide a critique of what VAL has suggested and how its suggestion is suitable for your purpose.

No other Gen AI tool is allowed to be used in this assignment.

More information about Val can be found here:

[Val GenAI Chatbot – Your virtual learning assistant - RMIT University](#)

More info how to properly reference AI tools can be found here:

[Citing and referencing guidelines for AI tools - Artificial intelligence \(AI\): acknowledgement and referencing guidelines - All guides at RMIT University](#)

8. Academic Integrity and Plagiarism (Standard Warning)

Academic integrity is about honest presentation of your academic work. It means acknowledging the work of others whilst developing your own insights, knowledge, and ideas. You should take extreme care that you have:

- Acknowledged words, data, diagrams, models, frameworks and/or ideas of others you have quoted (i.e., directly copied), summarized, paraphrased, discussed, or mentioned in your assessment through the appropriate referencing methods.
- Provided a reference list of the publication details so your readers can locate the source if necessary. This includes material taken from internet sites.

If you do not acknowledge the sources of your material, you may be accused of plagiarism because you have passed off the work and ideas of another person without appropriate reference, as if they were your own.

RMIT University treats plagiarism as a very serious offence constituting misconduct. Plagiarism covers a variety of inappropriate behaviors, including:

- Failure to properly document a source
- Copyright material from the internet or databases
- Collusion between students

In the case of suspected cases of academic misconduct, the academic staff who assess the student's work may add another assessment which is oral defense to determine student's genuine understanding of the submitted work.

For further information on our policies and procedures, please refer to the University website ([link](#)).

9. Assessment Declaration:

When you submit work electronically, you agree to the assessment declaration:
<https://www.rmit.edu.au/students/my-course/assessment-results/assessment>

More importantly, the following is applicable in this assignment.

Important: Condition 3 Bounded Process AI Declaration required

This assessment uses Condition 3: Bounded Process AI.

You may use AI for the approved process-based purposes listed below:

Researching for the appropriate algorithm, exploring options, and checking logic to be implemented to solve requirements labeled as "Special challenge" in Part 2 of the assignment specification.

You must not use AI beyond these approved purposes. Unless this assessment explicitly allows it, you must not use AI to produce the final answer, final analysis, final argument, final code, final design decisions, final reflection, or other final assessable content.

You remain responsible for all ideas, decisions, reasoning, content, and evidence submitted for marking. AI output must not replace your own thinking, judgement or authorship.

You must **submit the Condition 3 Bounded Process AI Declaration** with your assessment, **even if you did not use AI. If you did not use AI, complete the section confirming that no AI was used.**

If you used AI for approved process support, your declaration must identify the tool used, explain what it was used for, describe the AI output you received, explain what you accepted, rejected or revised, and describe how the AI interaction influenced your final submission.

Your assessment submission is not complete unless it includes both:

- your assessment work
- your completed Condition 3 Bounded Process AI Declaration

If you submit your assessment work without the required declaration, the whole assessment will be treated as incomplete. The submission date will be the date on which both the assessment work and the completed declaration are submitted.

Please complete the declaration carefully, accurately, and in full. It is part of the evidence used to confirm that your submission meets the AI condition for this assessment. If your declaration is incomplete, unclear, inconsistent with your submitted work, or raises concerns about the nature or extent of AI use, you may be asked to attend a follow-up conversation or oral authentication session. Where there is evidence that AI has been used beyond the permitted scope, or that AI use has not been declared accurately, the matter may be referred for academic integrity review.